Task 2: Do Bigrams Capture Additional Information?

Daniel Nickolov

Objectives

- Perform a secondary tokenization of the data to obtain bigrams
- Fit a logistic principal component regression model to the word-tokenized data

Setup

```
knitr::opts_knit$set(root.dir = normalizePath(".."))

library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.1     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.3     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(tidymodels)
```

```
## -- Attaching packages --------------------------------------- tidymodels 1.5.0 --
## v broom        1.0.13     v rsample      1.3.2
## v dials        1.4.4      v tailor       0.1.0
## v infer        1.1.0      v tune         2.1.0
## v modeldata    1.6.0      v workflows    1.3.0
## v parsnip      1.6.1      v workflowsets 1.1.1
## v recipes      1.4.0      v yardstick    1.4.0
## -- Conflicts ------------------------------------------ tidymodels_conflicts() --
## x scales::discard() masks purrr::discard()
## x dplyr::filter()   masks stats::filter()
## x recipes::fixed()  masks stringr::fixed()
## x dplyr::lag()      masks stats::lag()
## x yardstick::spec() masks readr::spec()
## x recipes::step()   masks stats::step()
```

```r
library(tokenizers)
library(textstem)
```

```
## Loading required package: koRpus.lang.en
## Loading required package: koRpus
## Loading required package: sylly
## For information on available language packages for 'koRpus', run
##
##   available.koRpus.lang()
##
## and see ?install.koRpus.lang()
##
##
## Attaching package: 'koRpus'
##
## The following object is masked from 'package:readr':
##
##     tokenize
```

```r
library(stopwords)
library(textrecipes)

source("../scripts/preprocessing.R")
```

```
## Loading required package: tidytext
## Loading required package: rvest
##
## Attaching package: 'rvest'
##
## The following object is masked from 'package:readr':
##
##     guess_encoding
##
## Loading required package: qdapRegex
##
## Attaching package: 'qdapRegex'
##
## The following object is masked from 'package:dplyr':
##
##     explain
##
## The following object is masked from 'package:ggplot2':
##
##     %+%
```

```r
load("../data/claims-clean-example.RData") # I had too many memory issues so I'm using the example clea

claims_df <- claims_clean %>%
  filter(bclass %in% c("Relevant claim content", "N/A: No relevant content.")) %>%
  mutate(
    doc_id = row_number(),
    claim_target = factor(if_else(bclass == "Relevant claim content", 1, 0), levels = c("1", "0")) # ma
```

```
  ) %>%
  select(doc_id, text_clean, claim_target)
```

Tokenization

Fitting to Word-Tokenized Data (Unigrams)

```
set.seed(111126)
unigram_recipe <- recipe(claim_target ~ text_clean + doc_id, data = claims_df) %>%
  update_role(doc_id, new_role = "ID") %>%
  step_tokenize(text_clean, token = "words") %>%
  step_stopwords(text_clean) %>%
  step_tokenfilter(text_clean, max_tokens = 1000) %>%
  step_tf(text_clean) %>%
  step_normalize(all_predictors()) %>%
  step_pca(all_predictors(), num_comp = 20, prefix = "unigram_PC")

unigram_prep <- prep(unigram_recipe)
unigram_pcs <- bake(unigram_prep, new_data = NULL)

fit_unigram <- glm(claim_target ~ . - doc_id, data = unigram_pcs, family = binomial)
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
claims_df$log_odds_unigram <- predict(fit_unigram, newdata = unigram_pcs, type = "link")
```

Fitting Bigrams

```
set.seed(111126)
bigram_recipe <- recipe(~ text_clean + doc_id, data = claims_df) %>%
  update_role(doc_id, new_role = "ID") %>%
  step_tokenize(text_clean, token = "ngrams", options = list(n = 2)) %>%
  step_stopwords(text_clean) %>%
  step_tokenfilter(text_clean, max_tokens = 1000) %>%
  step_tf(text_clean) %>%
  step_normalize(all_predictors()) %>%
  step_pca(all_predictors(), num_comp = 10, prefix = "bigram_PC") # K = 10 bigram principle components

bigram_prep <- prep(bigram_recipe)
bigram_pcs <- bake(bigram_prep, new_data = NULL)
```

Now, we can stack the unigram and bigram PCs into one model and run an ANOVA test to determine if the bigrams do capture additional information.

Setup

```r
stacked_df <- claims_df %>%
  select(doc_id, claim_target, log_odds_unigram) %>%
  inner_join(bigram_pcs, by = "doc_id") %>%
  select(-doc_id)

fit_base <- glm(claim_target ~ log_odds_unigram, data = stacked_df, family = binomial)
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```r
fit_stacked <- glm(claim_target ~ ., data = stacked_df, family = binomial)
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

Running the ANOVA

```r
lrt_result <- anova(fit_base, fit_stacked, test = "Chisq")
print(lrt_result)
```

```
## Analysis of Deviance Table
##
## Model 1: claim_target ~ log_odds_unigram
## Model 2: claim_target ~ log_odds_unigram + bigram_PC01 + bigram_PC02 +
##     bigram_PC03 + bigram_PC04 + bigram_PC05 + bigram_PC06 + bigram_PC07 +
##     bigram_PC08 + bigram_PC09 + bigram_PC10
##   Resid. Df Resid. Dev Df Deviance Pr(>Chi)
## 1      2138     2514.8
## 2      2128     2510.6 10   4.2207   0.9368
```

Running the test, we get a p-value of 0.9368418, which is well above a standard level of significance of $\alpha = 0.05$, meaning that when comparing unigram-tokenized data to bigram-tokenized data in this dataset, there isn't a significant difference in the information captured about a page.